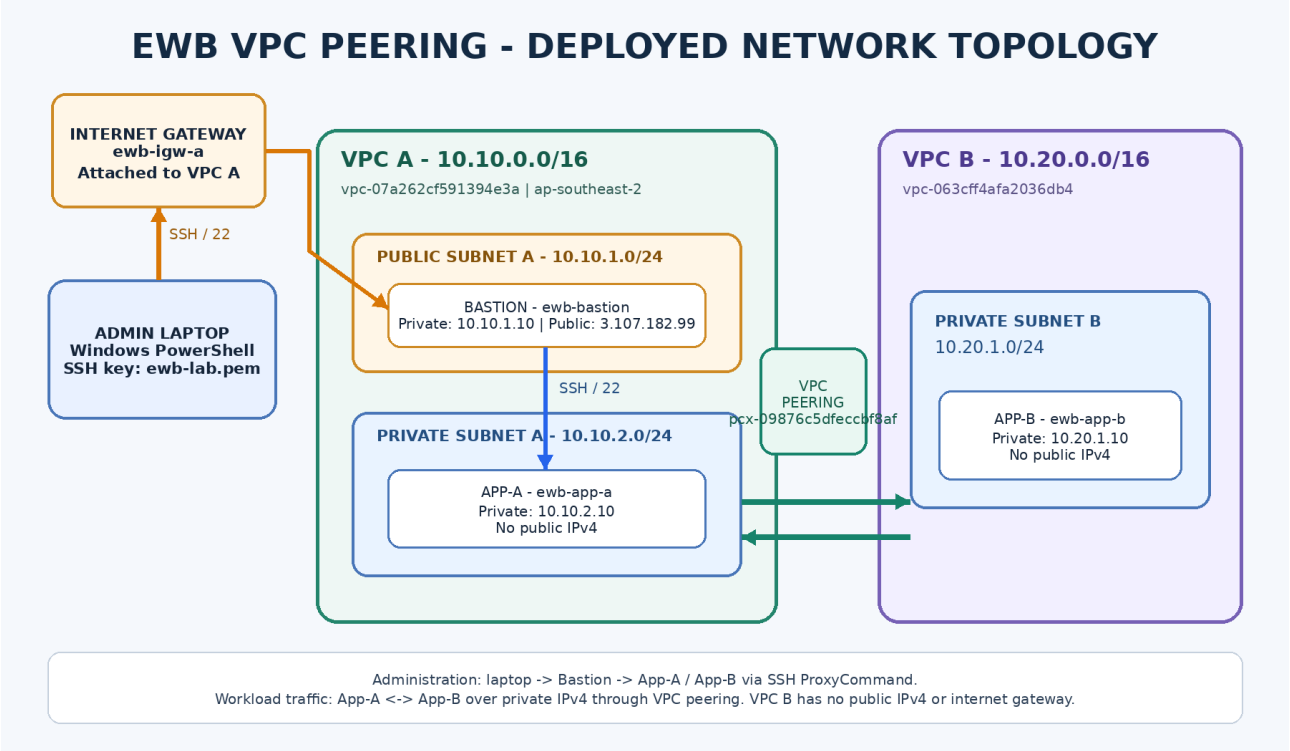
EWB CLOUD LABS

AWS VPC PEERING

Private connectivity between two Amazon VPCs

PROJECT IMPLEMENTATION & VALIDATION REPORT

BUILD / CONNECT / TEST / EXPLAIN



PROJECT DETAILS	VALUE
Student	Shekar
Project	EWB-VPC-Peering
AWS Region	Asia Pacific (Sydney) - ap-southeast-2
Availability Zone	ap-southeast-2a
Completion evidence captured	7 November 2026

Prepared from the EWB Courses Student Project Workbook and the AWS console / terminal evidence captured during implementation.

1. Executive Summary

This project implements a private network connection between two Amazon Virtual Private Clouds (VPCs) in the same AWS account and Region. VPC A hosts a public bastion instance and a private application instance. VPC B hosts a second private application instance. A VPC peering connection and reciprocal route-table entries enable the two private workloads to communicate using their private IPv4 addresses.

Outcome verified

All three EC2 instances reached the Running state and showed 3/3 status checks passed in the final Instances view. SSH access from the Windows laptop to the bastion and onward to both private instances was demonstrated. ICMP ping succeeded in both directions between App-A and App-B, with 4/4 replies and 0% packet loss in each test.

MEASURE	OBSERVED RESULT
VPCs	2 non-overlapping VPCs
EC2 instances	3 (Bastion, App-A, App-B)
VPC peering	1 connection, status Active
App-B to App-A ping	4 received / 4 sent; 0% packet loss
App-A to App-B ping	4 received / 4 sent; 0% packet loss
Private workload public IPv4	None for App-A and App-B

1.1 Scope and objectives

- Create two VPCs with non-overlapping IPv4 CIDR blocks.
- Create a public subnet for the bastion in VPC A and private subnets for both application workloads.
- Attach an internet gateway to VPC A and configure the public subnet default route.
- Establish an active VPC peering connection and add forward and return routes.
- Restrict inbound SSH and ICMP traffic to the required host addresses.
- Demonstrate private SSH administration through the bastion and bidirectional private IP connectivity between App-A and App-B.

1.2 Environment

ITEM	OBSERVED VALUE
AWS Region	ap-southeast-2 (Sydney)
Availability Zone	ap-southeast-2a
Operating system	Amazon Linux 2023
Instance type	t3.micro on all three instances
Key pair	ewb-lab (RSA, PEM)
Root storage	8 GiB EBS gp3 per instance

2. Architecture and Address Plan

The design follows the EWB workbook topology: the laptop reaches the bastion over SSH, and the bastion opens separate SSH sessions to the private application instances. The workloads exchange traffic directly over private IPv4 addresses through the peering connection. VPC B has no internet gateway, and the application instances have no public IPv4 addresses.

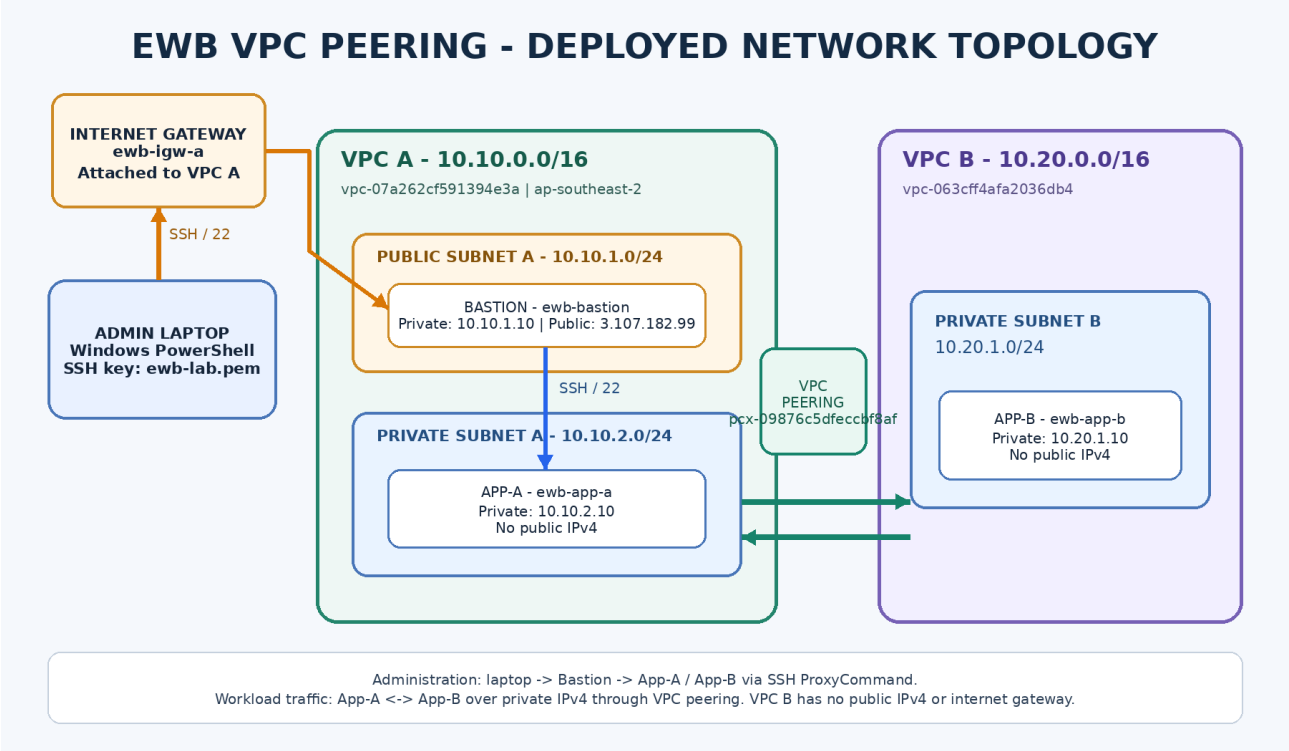


Figure 1. As-built logical topology. Public and private IP values are taken from the AWS console evidence.

2.1 VPC and subnet address plan

RESOURCE	VPC	CIDR	SUBNET ID / NOTES
ewb-vpc-a	VPC A	10.10.0.0/16	vpc-07a262cf591394e3a
ewb-a-public	VPC A	10.10.1.0/24	subnet-0d4417bc09da303ae
ewb-a-private	VPC A	10.10.2.0/24	subnet-09464044a9bc81a92
ewb-vpc-b	VPC B	10.20.0.0/16	vpc-063cff4afa2036db4
ewb-b-private	VPC B	10.20.1.0/24	subnet-0751251d392b846f4

2.2 Instance address plan

INSTANCE	SUBNET	PRIVATE IPv4	PUBLIC IPv4	INSTANCE ID
ewb-bastion	ewb-a-public	10.10.1.10	3.107.182.99	i-0ccfb0a30b2413c9b
ewb-app-a	ewb-a-private	10.10.2.10	None	i-0be936f3f7e725846
ewb-app-b	ewb-b-private	10.20.1.10	None	i-0cfd701e69b055033

CIDR design note

VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16) do not overlap. Each /24 subnet is contained within its parent VPC CIDR. This non-overlap is required for VPC peering routes to work.

3. AWS Resource Inventory

RESOURCE	NAME	RESOURCE ID / VALUE
VPC A	ewb-vpc-a	vpc-07a262cf591394e3a
VPC B	ewb-vpc-b	vpc-063cff4afa2036db4
Internet gateway	ewb-igw-a	igw-00d4e34dfef30b3be - Attached to VPC A
VPC peering	ewb-a-b-peer	pcx-09876c5dfeccbf8af - Active
Public route table	ewb-rt-a-public	rtb-020862505dff44264
VPC A private route table	ewb-rt-a-private	rtb-0630f0a97f37aae58
VPC B private route table	ewb-rt-b-private	rtb-085804ff852827a6d
Bastion security group	ewb-sg-bastion	sg-00984bfc34149ba38
App-A security group	ewb-sg-app-a	sg-0415499bb0e3d8e42
App-B security group	ewb-sg-app-b	sg-05e6acaa9923b79fd
EC2 key pair	ewb-lab	RSA; PEM private key stored locally

Resources were tagged with Project = EWB-VPC-Peering and Student = Shekar where the console screenshots show tags. The AWS account owner ID is intentionally not reproduced in this report.

3.1 Internet gateway

The internet gateway named ewb-igw-a was created and attached to ewb-vpc-a. The public route table uses this gateway as the target for 0.0.0.0/0. The gateway is not attached to VPC B.

3.2 VPC peering connection

FIELD	OBSERVED VALUE
Name	ewb-a-b-peer
Peering connection ID	pcx-09876c5dfeccbf8af
Requester VPC	vpc-07a262cf591394e3a / ewb-vpc-a
Requester CIDR	10.10.0.0/16
Accepter VPC	vpc-063cff4afa2036db4 / ewb-vpc-b
Accepter CIDR	10.20.0.0/16
Region	Sydney (ap-southeast-2)
Status	Active

4. Route Table Configuration

Three route tables were created and explicitly associated with their intended subnets. The AWS-created local route was retained in each table. The two private route tables contain only their local VPC route and the route to the remote VPC over the peering connection. The public route table also contains the internet default route to the internet gateway.

ROUTE TABLE	SUBNET ASSOCIATION	DESTINATION	TARGET	STATUS
ewb-rt-a-public	ewb-a-public	10.10.0.0/16	local	Active
ewb-rt-a-public	ewb-a-public	0.0.0.0/0	igw-00d4e34dfef30b3be	Active
ewb-rt-a-public	ewb-a-public	10.20.0.0/16	pcx-09876c5dfeccbf8af	Active
ewb-rt-a-private	ewb-a-private	10.10.0.0/16	local	Active
ewb-rt-a-private	ewb-a-private	10.20.0.0/16	pcx-09876c5dfeccbf8af	Active
ewb-rt-b-private	ewb-b-private	10.20.0.0/16	local	Active
ewb-rt-b-private	ewb-b-private	10.10.0.0/16	pcx-09876c5dfeccbf8af	Active

4.1 Why the three peering routes are needed

- App-A to App-B: the VPC A private route table sends 10.20.0.0/16 to the peering connection.
- App-B to App-A: the VPC B private route table sends 10.10.0.0/16 to the peering connection.
- Bastion to App-B over SSH ProxyCommand: the VPC A public route table also sends 10.20.0.0/16 to the peering connection. The route is more specific than the 0.0.0.0/0 internet default route.

Routing result

The final AWS console screenshots show all listed routes with Active status. Bidirectional ICMP results later in this report demonstrate that both forward and return routing paths were usable during the test.

5. Security Group Configuration

The security groups restrict inbound traffic to the administrative bastion and the two application hosts. SSH uses TCP port 22. ICMP rules permit Echo Request from the opposite application host for the peering connectivity tests. Default outbound allow-all rules were retained, consistent with the beginner-lab workbook.

SECURITY GROUP	INBOUND TYPE	PROTOCOL / PORT	SOURCE	PURPOSE
ewb-sg-bastion	SSH	TCP / 22	103.135.64.238/32	Administrative access from the public IP shown at setup time
ewb-sg-app-a	SSH	TCP / 22	10.10.1.10/32	SSH from Bastion
ewb-sg-app-a	ICMP Echo Request	ICMPv4	10.20.1.10/32	Ping from App-B
ewb-sg-app-b	SSH	TCP / 22	10.10.1.10/32	SSH from Bastion
ewb-sg-app-b	ICMP Echo Request	ICMPv4	10.10.2.10/32	Ping from App-A

Bastion source IP caveat

The bastion security group screenshot shows SSH allowed from 103.135.64.238/32 at the time it was configured. If the laptop public IPv4 changes, update this /32 source to the current trusted public IPv4 before a later SSH session. Do not broaden SSH to 0.0.0.0/0.

5.1 Security posture

- Only the bastion has a public IPv4 address. App-A and App-B have no public IPv4 addresses.
- App-A and App-B allow SSH only from the bastion private address 10.10.1.10/32.
- ICMP Echo Request rules are limited to the other private application host /32.
- The same local RSA PEM key pair, ewb-lab, was selected for all three instances. The private key remained on the Windows laptop; it was not copied to the EC2 instances.

6. EC2 Deployment

Three EC2 instances were launched in the Sydney Region using Amazon Linux 2023, 64-bit x86 architecture and t3.micro instance type. Each instance used the ewb-lab key pair and an 8 GiB gp3 root EBS volume with Delete on termination set to Yes. The final Instances screenshot showed all three instances in Running state with 3/3 status checks passed.

INSTANCE	INSTANCE ID	VPC / SUBNET	PRIVATE IPv4	PUBLIC IPv4	STATE
ewb-bastion	i-0ccfb0a30b2413c9b	VPC A / public	10.10.1.10	3.107.182.99	Running
ewb-app-a	i-0be936f3f7e725846	VPC A / private	10.10.2.10	None	Running
ewb-app-b	i-0cfd701e69b055033	VPC B / private	10.20.1.10	None	Running

6.1 Bastion access path

The bastion is the only instance with a public IPv4 address. SSH from the Windows laptop uses the local key at %USERPROFILE%\.ssh\ewb-lab.pem and the Amazon Linux username ec2-user. The private application instances are accessed through the bastion using an SSH ProxyCommand.

7. SSH Connectivity Tests

SSH connectivity was demonstrated from the Windows PowerShell client. The PEM file permissions initially prevented OpenSSH from loading the key. Inheritance was removed from the key file and read access was granted to the local Windows account only. SSH to the bastion then succeeded.

7.1 Laptop to Bastion

```
ssh -o IdentitiesOnly=yes -i "$env:USERPROFILE\.ssh\ewb-lab.pem" ec2-user@3.107.182.99
```

Observed shell prompt: [ec2-user@ip-10-10-1-10 ~]$. The hostname command returned ip-10-10-1-10.ap-southeast-2.compute.internal and whoami returned ec2-user.

7.2 Laptop to App-A through Bastion

```
$key = "$env:USERPROFILE\.ssh\ewb-lab.pem"
ssh -i "$key" -o IdentitiesOnly=yes -o "ProxyCommand=ssh -i $key -o IdentitiesOnly=yes -W %h:%p ec2-user@3.107.182.99" ec2-user@10.10.2.10
```

Observed shell prompt: [ec2-user@ip-10-10-2-10 ~]$. This confirms access to the private App-A address through the bastion.

7.3 Laptop to App-B through Bastion

```
ssh -i "$key" -o IdentitiesOnly=yes -o "ProxyCommand=ssh -i $key -o IdentitiesOnly=yes -W %h:%p ec2-user@3.107.182.99" ec2-user@10.20.1.10
```

Observed shell prompt: [ec2-user@ip-10-20-1-10 ~]$. The hostname command returned ip-10-20-1-10.ap-southeast-2.compute.internal and whoami returned ec2-user.

Key-handling note

The private key remains on the client computer. The SSH ProxyCommand forwards the TCP stream through the bastion; it does not require copying the PEM file to the bastion or either private host.

8. VPC Peering Connectivity Validation

The project was validated with ICMP Echo Request tests in both directions, using the private IPv4 addresses. Both tests returned four replies with 0% packet loss.

TEST	SOURCE	DESTINATION	PACKETS	PACKET LOSS	AVG RTT
App-B -> App-A	10.20.1.10	10.10.2.10	4/4 received	0%	0.267 ms
App-A -> App-B	10.10.2.10	10.20.1.10	4/4 received	0%	0.254 ms

8.1 App-B to App-A

```
ping -c 4 10.10.2.10
```

Observed: 4 packets transmitted, 4 received, 0% packet loss. RTT min/avg/max/mdev = 0.259/0.267/0.280/0.007 ms.

8.2 App-A to App-B

```
ping -c 4 10.20.1.10
```

Observed: 4 packets transmitted, 4 received, 0% packet loss. RTT min/avg/max/mdev = 0.247/0.254/0.261/0.006 ms.

Validation conclusion

The observed bidirectional ping results are consistent with working forward and return routes across the active VPC peering connection. The SSH sessions also demonstrate that the private hosts can be administered through the bastion without public IPv4 addresses on the application instances.

9. Remaining Workbook Evidence and Cleanup

The EWB workbook asks for two additional negative-test demonstrations and cleanup evidence. These were not shown in the screenshots available for this report, so they are deliberately marked as not evidenced rather than described as completed.

WORKBOOK ITEM	EVIDENCE STATUS	ACTION TO COMPLETE
Baseline ping failure before peering / peer routes	Not captured in the supplied evidence.	The test was intended before establishing peering and routes. Do not recreate a historical result. Ask the instructor whether a post-build route-isolation demonstration is acceptable if baseline evidence is mandatory.
Controlled ICMP failure and recovery	Not captured in the supplied evidence.	Temporarily remove only the App-B inbound ICMP Echo Request rule from 10.10.2.10/32; ping App-B from App-A; restore the rule; rerun the ping and capture the recovery.
Cleanup evidence	Not captured. The latest EC2 list shows all three instances Running.	After instructor verification, stop or terminate the lab instances as required; then remove remaining lab resources and verify Billing / resource lists.

Important before submission

The core positive connectivity implementation is demonstrated. If the evaluator follows every workbook criterion, complete the controlled ICMP failure/recovery test and add cleanup evidence before final submission. The pre-peering baseline is historical and was not captured before the peering routes were added.

9.1 Cleanup sequence from the workbook

1. Save all screenshots and confirm instructor review before deleting or stopping resources.
2. Terminate the three EC2 instances if the lab is complete; verify their lab EBS root volumes are deleted or remove leftover volumes and snapshots.
3. Delete the VPC peering connection and remove peering routes.
4. After network interfaces are removed, delete lab security groups, subnets and custom route tables.
5. Detach and delete ewb-igw-a, then delete VPC A and VPC B.
6. Check the Region for remaining chargeable resources, including EBS volumes, snapshots, public IPv4 allocations and any resources with the project tag.

Stopping EC2 instances reduces compute usage but does not remove EBS storage charges. Termination and cleanup should be performed only when the project no longer needs to remain available for demonstration.

10. Conclusion

The implemented design consists of two non-overlapping VPCs connected by an active VPC peering connection. VPC A provides the public bastion entry point and hosts App-A in a private subnet. VPC B hosts App-B in a private subnet and does not expose a public IPv4 address. Explicit subnet route-table associations and reciprocal peering routes provide private reachability between the workloads. Security group rules scope SSH access to the bastion private IP and ICMP Echo Request to the peer workload.

The final screenshots and terminal outputs demonstrate successful SSH access to the bastion, App-A and App-B, as well as bidirectional ping with 0% packet loss. The remaining workbook items are the historical baseline failure, controlled ICMP failure/recovery and cleanup evidence; these are identified as pending/not captured.

References and evidence basis

- EWB Courses, AWS VPC Peering - Student Project Workbook, Project 01, 14 pages, October 2026. This workbook defines the intended architecture, CIDRs, routes, security rules, validation steps, submission checklist and cleanup sequence.
- AWS console screenshots and Windows PowerShell / Amazon Linux terminal outputs supplied during the project conversation on 6-7 November 2026.
- The separate companion file EWB_VPC_Peering_All_Screenshots.docx contains the full screenshot collection assembled from the conversation. This report includes a selected evidence appendix.

Appendix A. Selected Implementation Evidence

The screenshots below are original captures supplied during the implementation. Captions describe the evidence visible in each capture. The complete screenshot collection is provided separately in EWB_VPC_Peering_All_Screenshots.docx.

VPC B created

The console confirms VPC B ewb-vpc-b, its VPC ID and IPv4 CIDR 10.20.0.0/16.

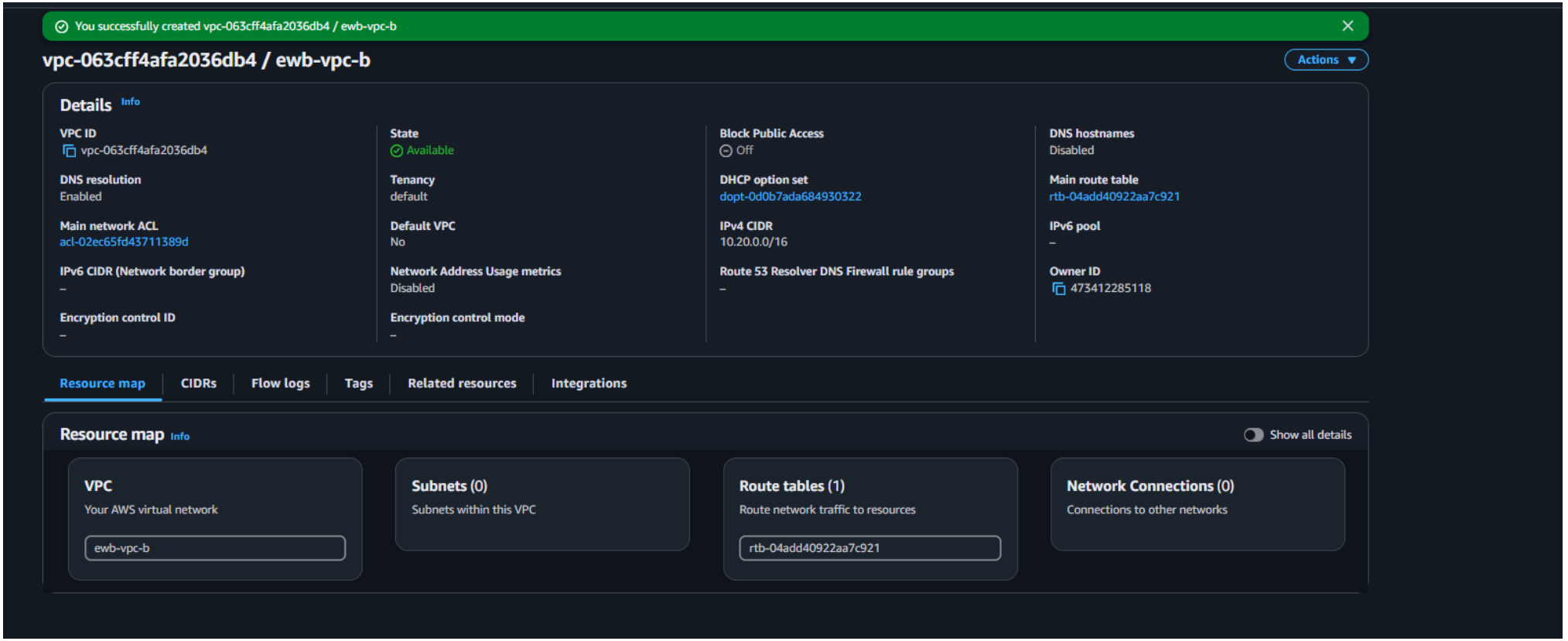


Figure evidence: image(20261106-130645).png

VPC A private subnet details

The console shows ewb-a-private, its 10.10.2.0/24 CIDR, VPC A association and subnet ID.

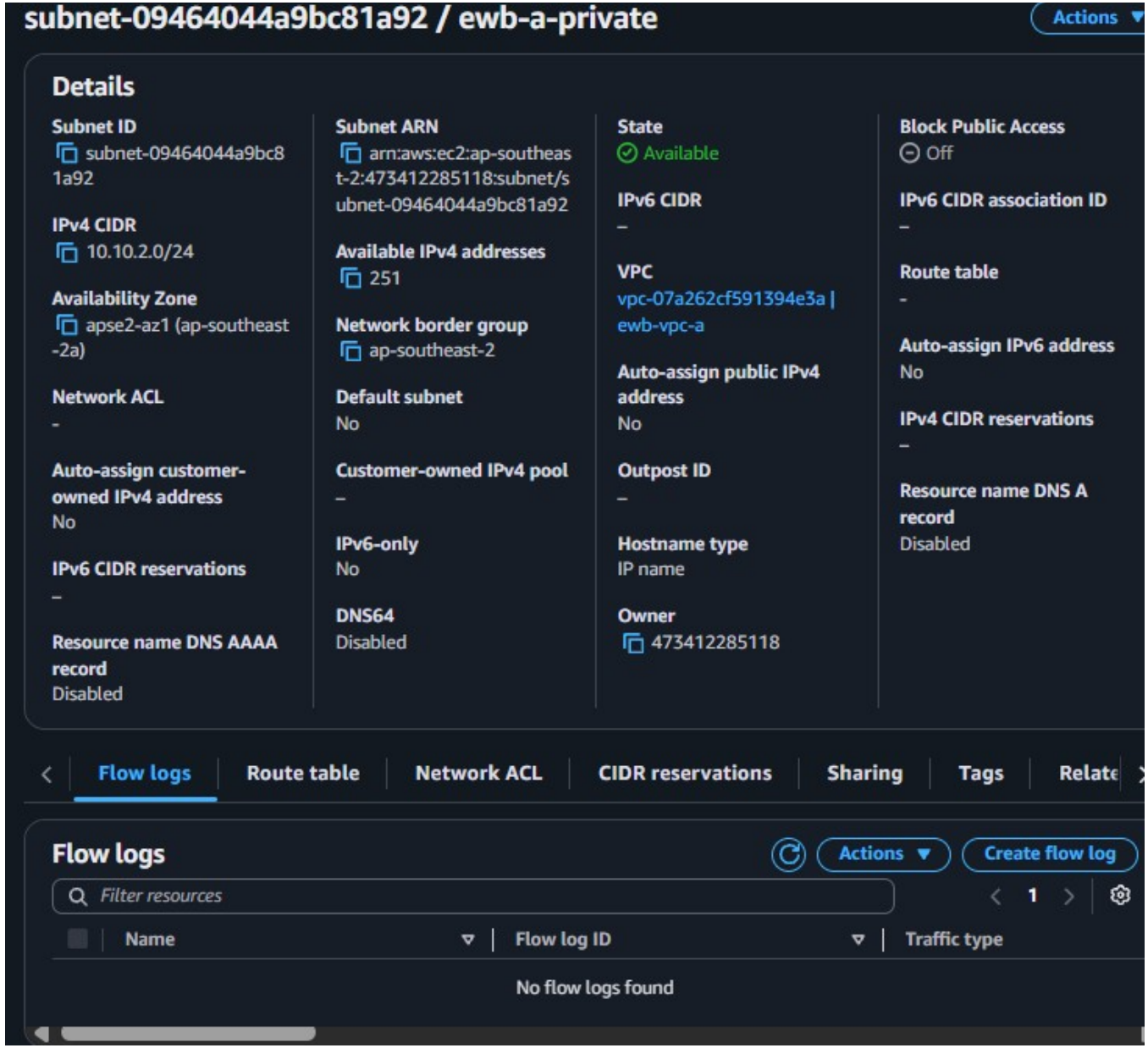


Figure evidence: image(20261106-122009).png

Internet gateway attached

The console confirms ewb-igw-a is attached to VPC A.

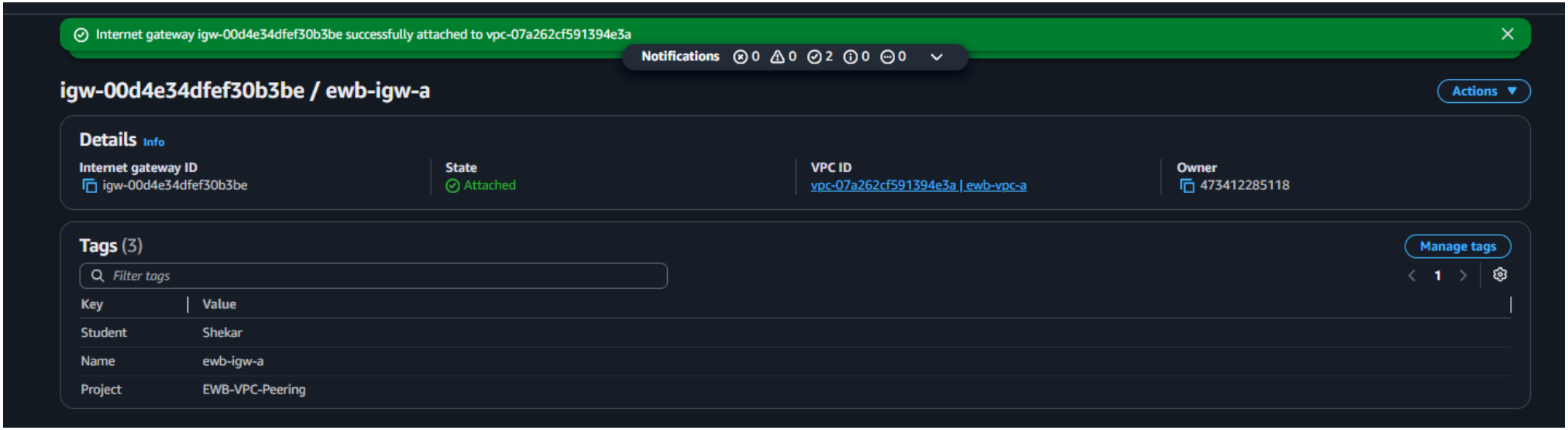


Figure evidence: image(20261106-122907).png

Active VPC peering connection

The peering connection ewb-a-b-peer is Active and identifies both VPCs and their CIDRs.

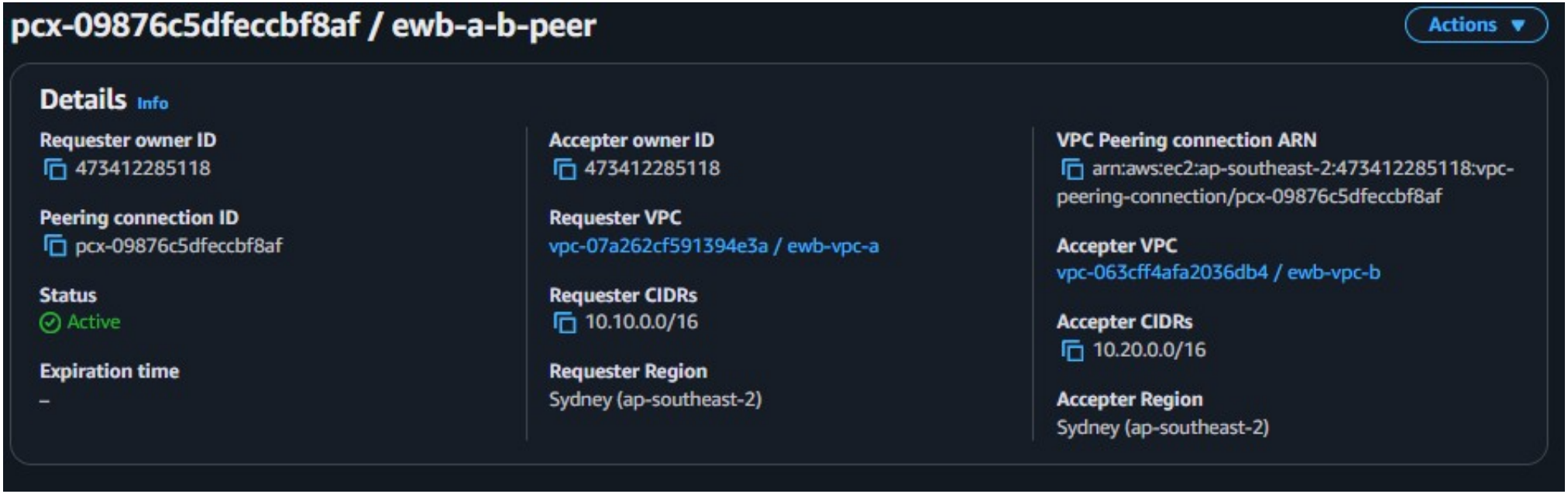


Figure evidence: image(20261106-135851).png

VPC A private route table

The private A table is explicitly associated with ewb-a-private and contains the local and peering routes.

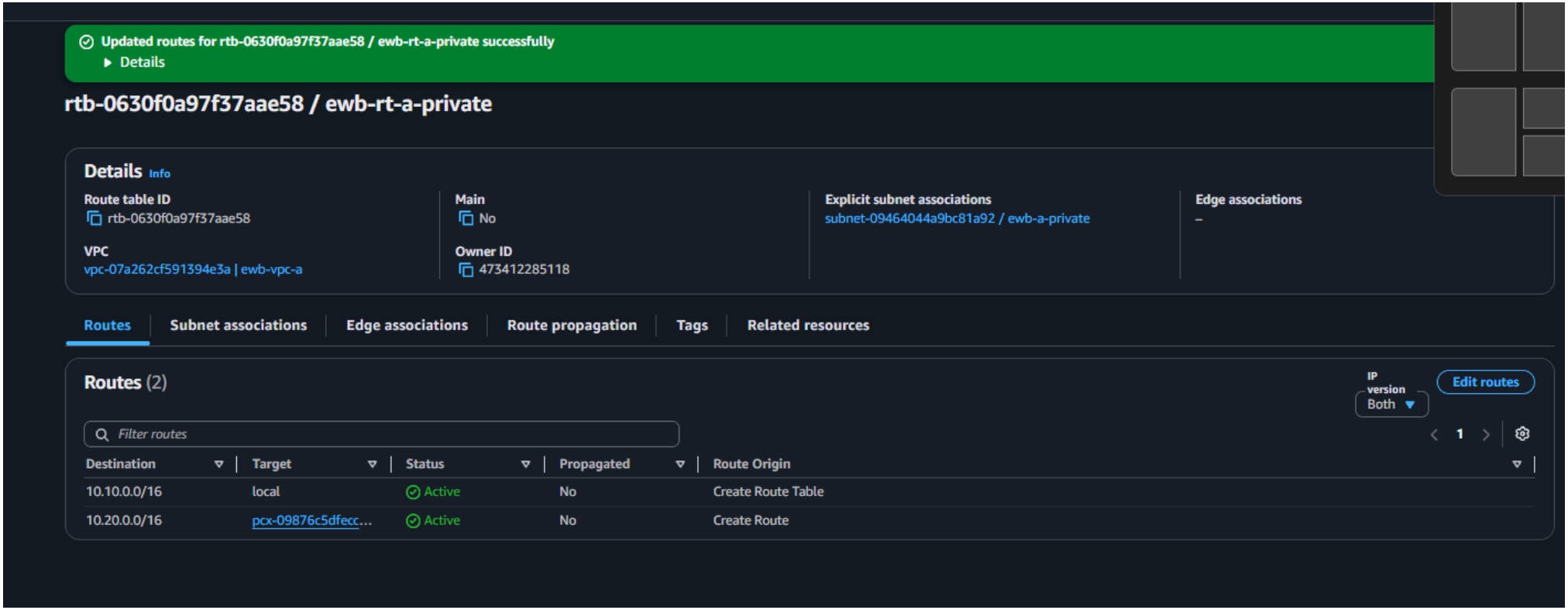


Figure evidence: image(20261106-140120).png

VPC B private route table

The private B table is explicitly associated with ewb-b-private and contains the local and return peering routes.

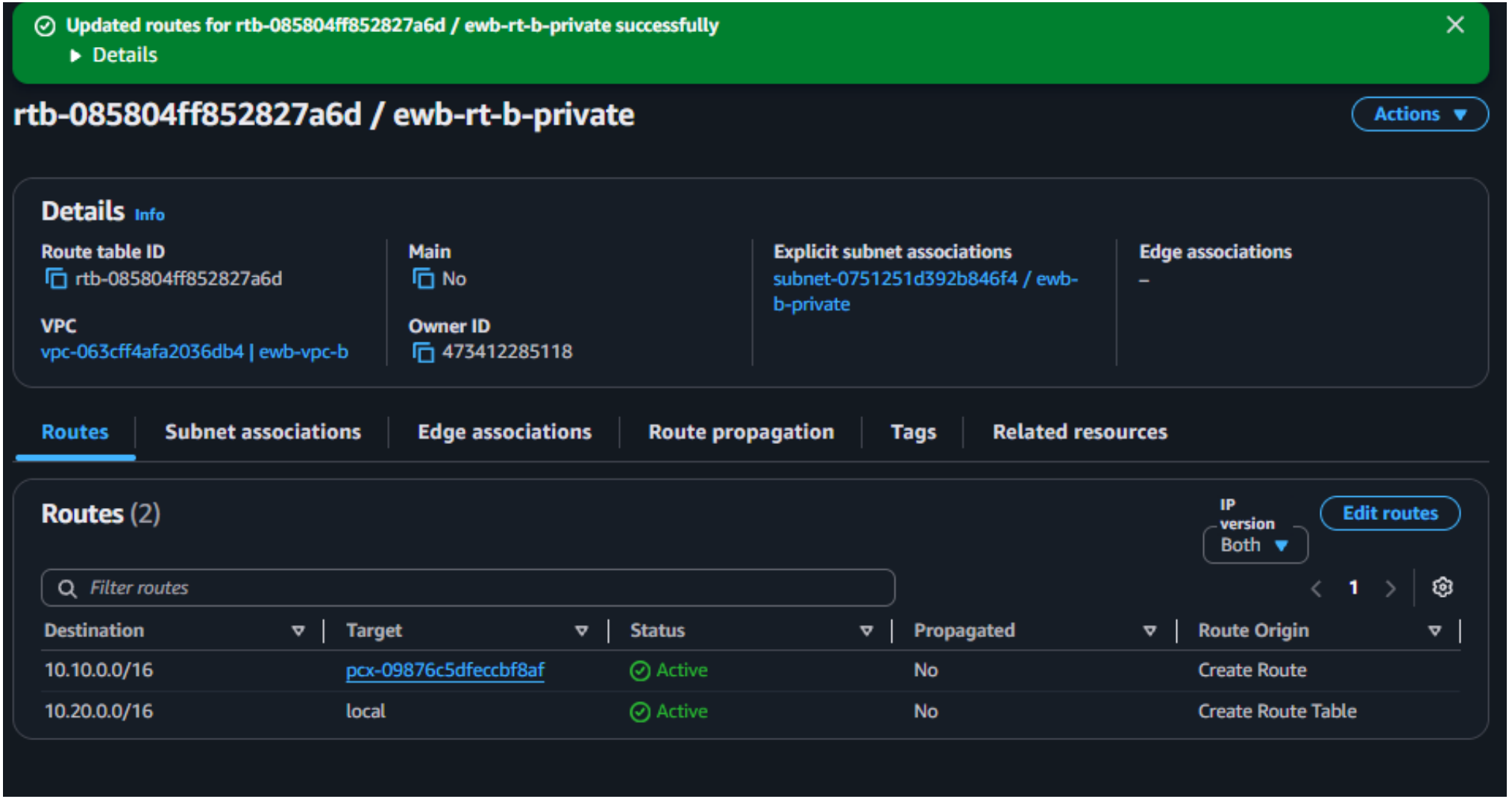


Figure evidence: image(20261106-140610).png

VPC A public route table

The public table is associated with ewb-a-public and contains local, internet gateway default and VPC B peering routes.

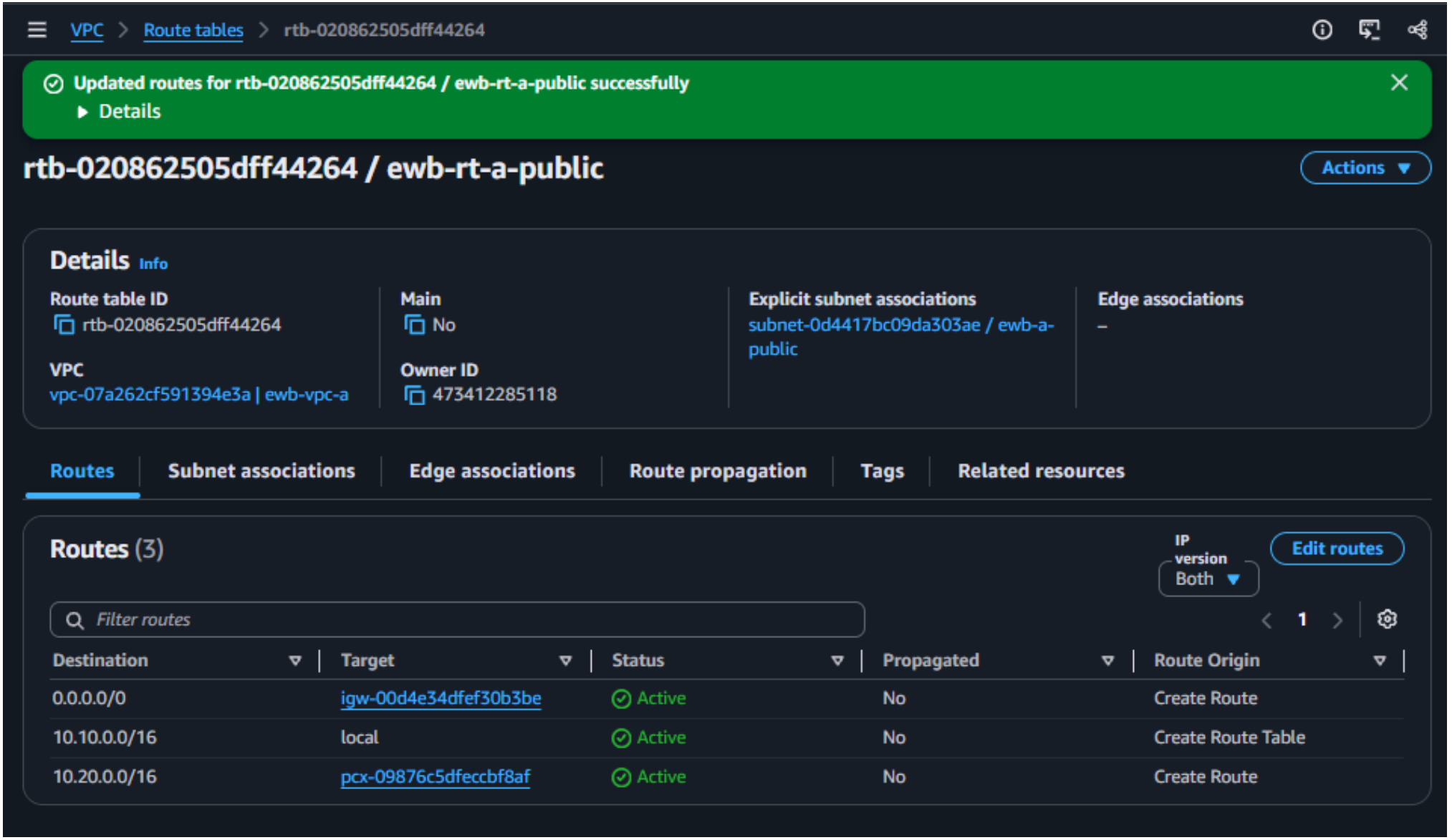


Figure evidence: image(20261106-140857).png

Bastion security group

The bastion security group shows SSH restricted to a single public IPv4 /32 source.

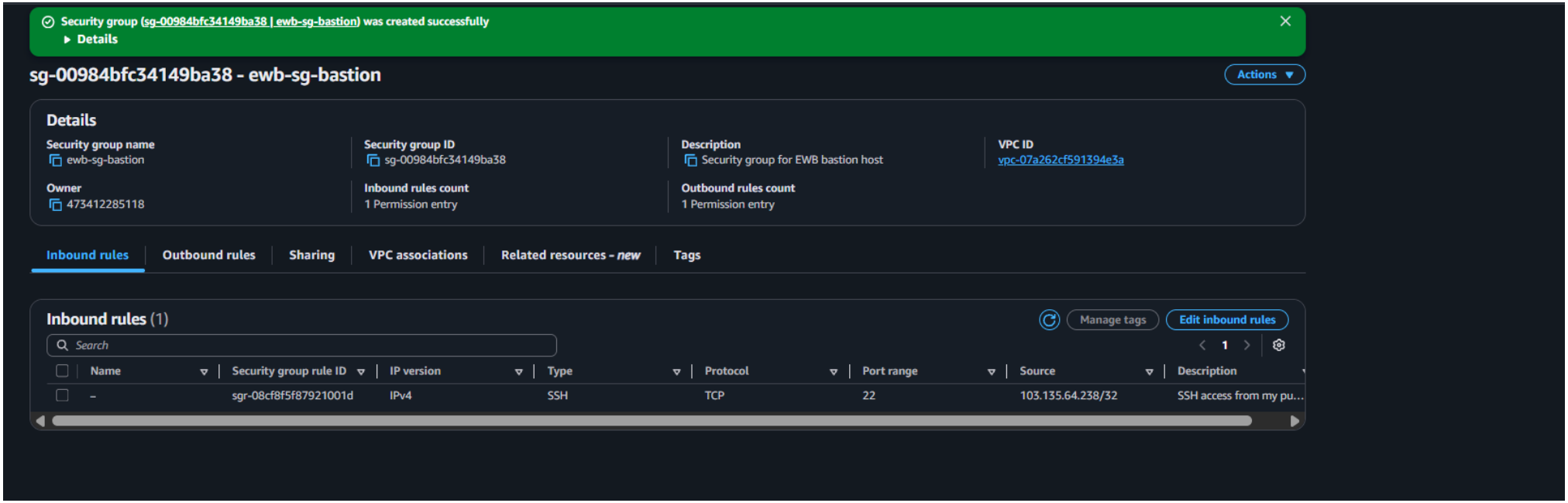


Figure evidence: image(20261106-141926).png

App-A security group

The App-A security group shows SSH from Bastion and ICMP Echo Request from App-B.

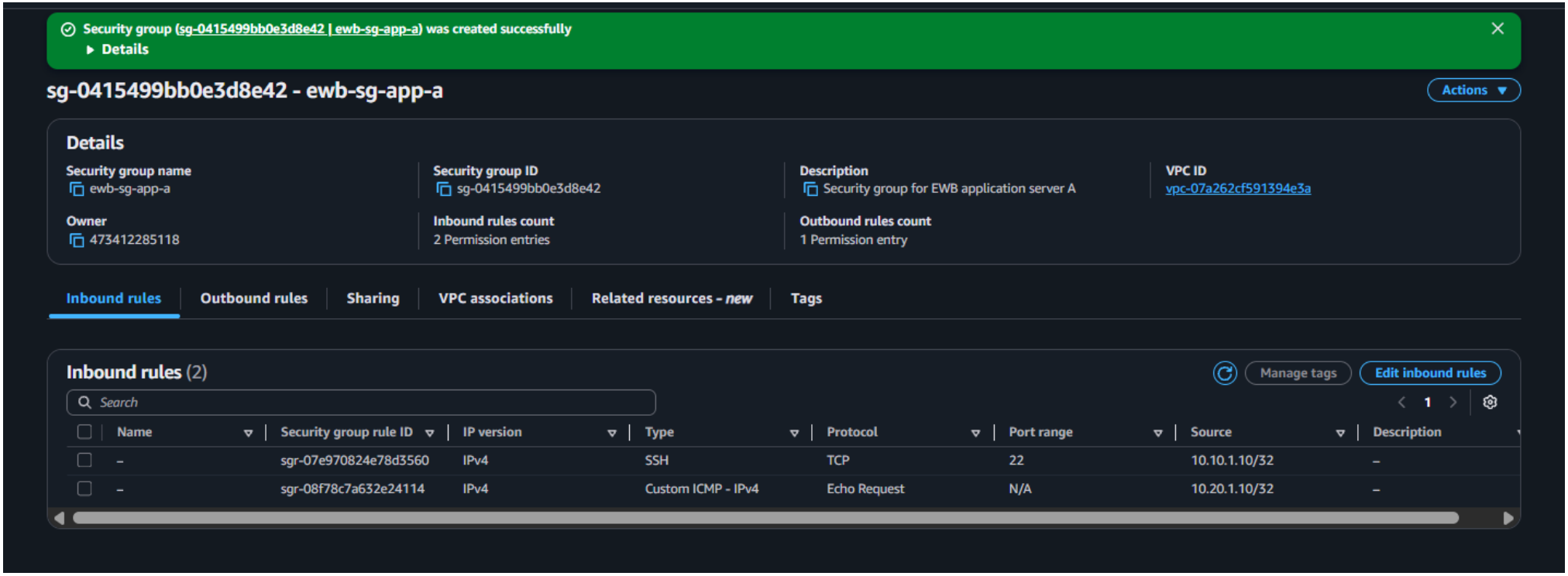


Figure evidence: image(20261106-142830).png

App-B security group

The App-B security group shows SSH from Bastion and ICMP Echo Request from App-A.

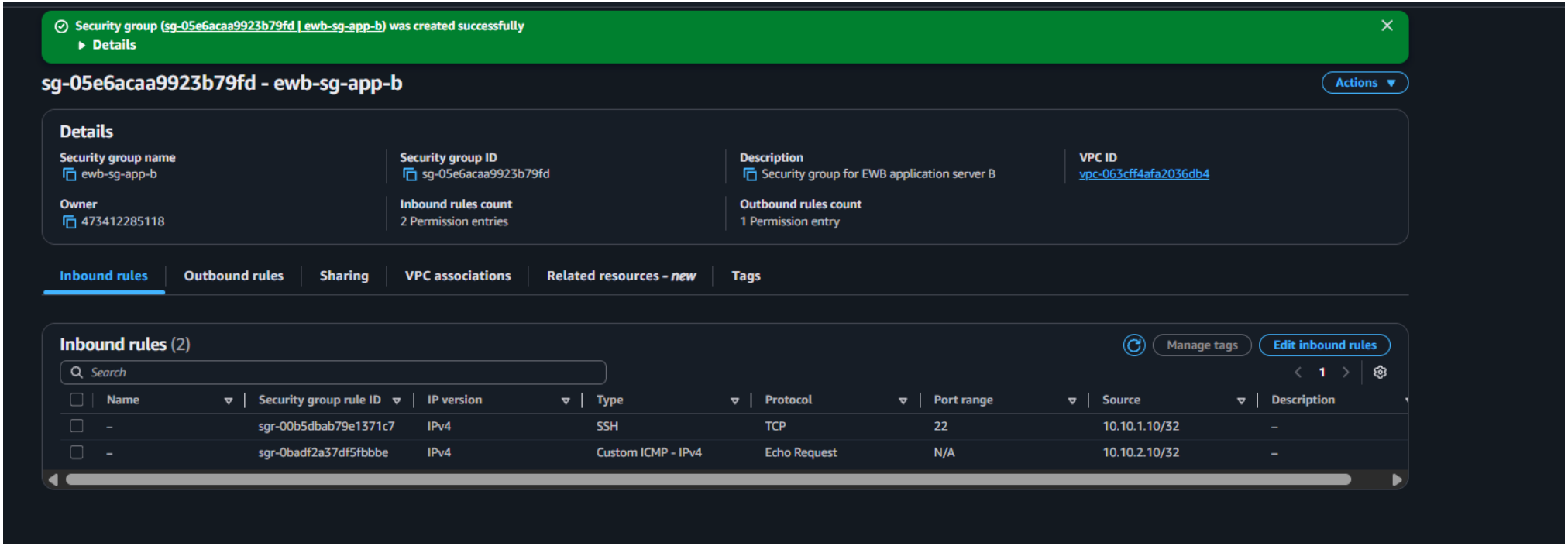


Figure evidence: image(20261106-143359).png

All three EC2 instances running

The final Instances view shows ewb-bastion, ewb-app-a and ewb-app-b Running with 3/3 checks passed.

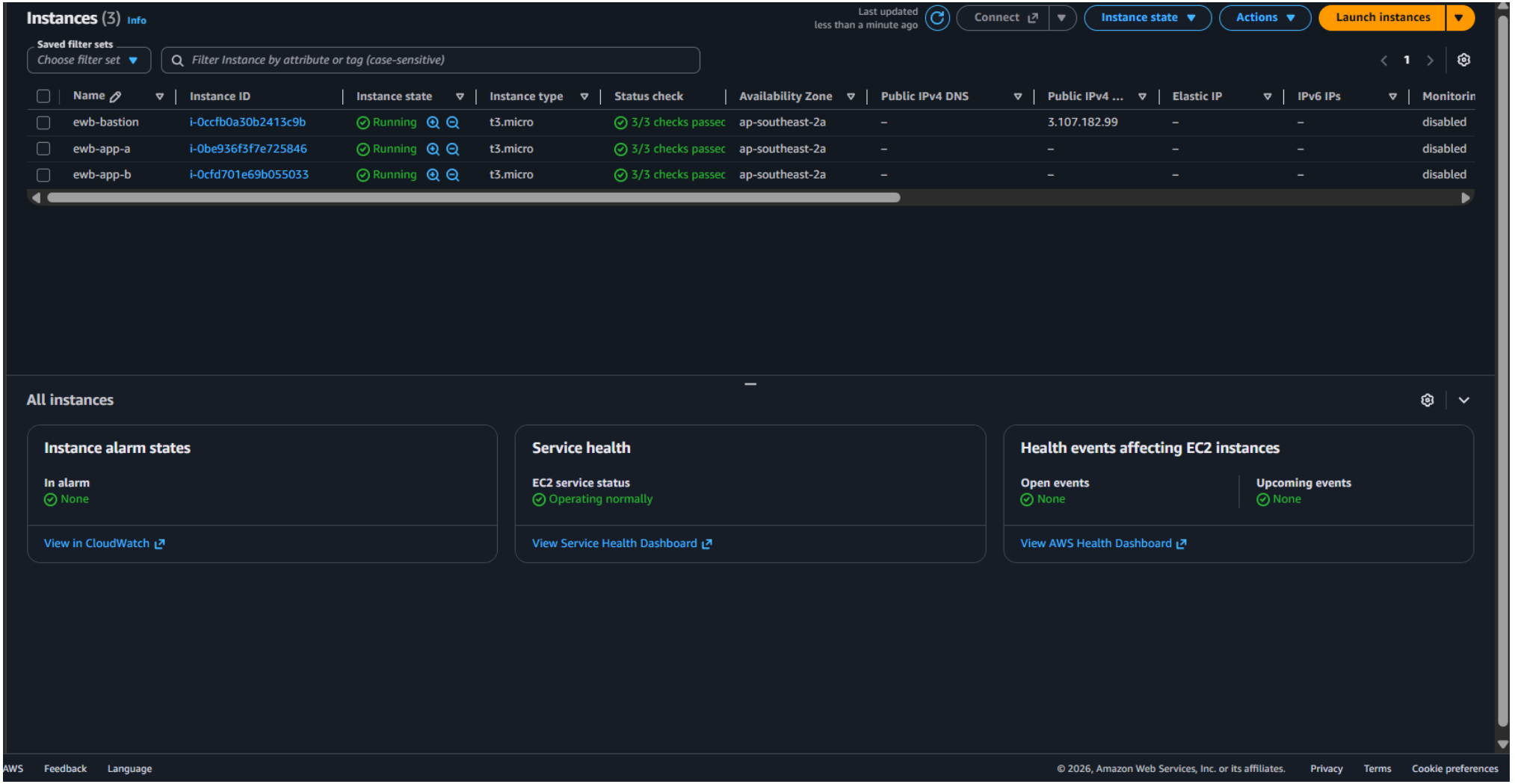


Figure evidence: image(20261107-113227).png

Bastion instance details

The selected Bastion instance shows private IP 10.10.1.10 and public IP 3.107.182.99.

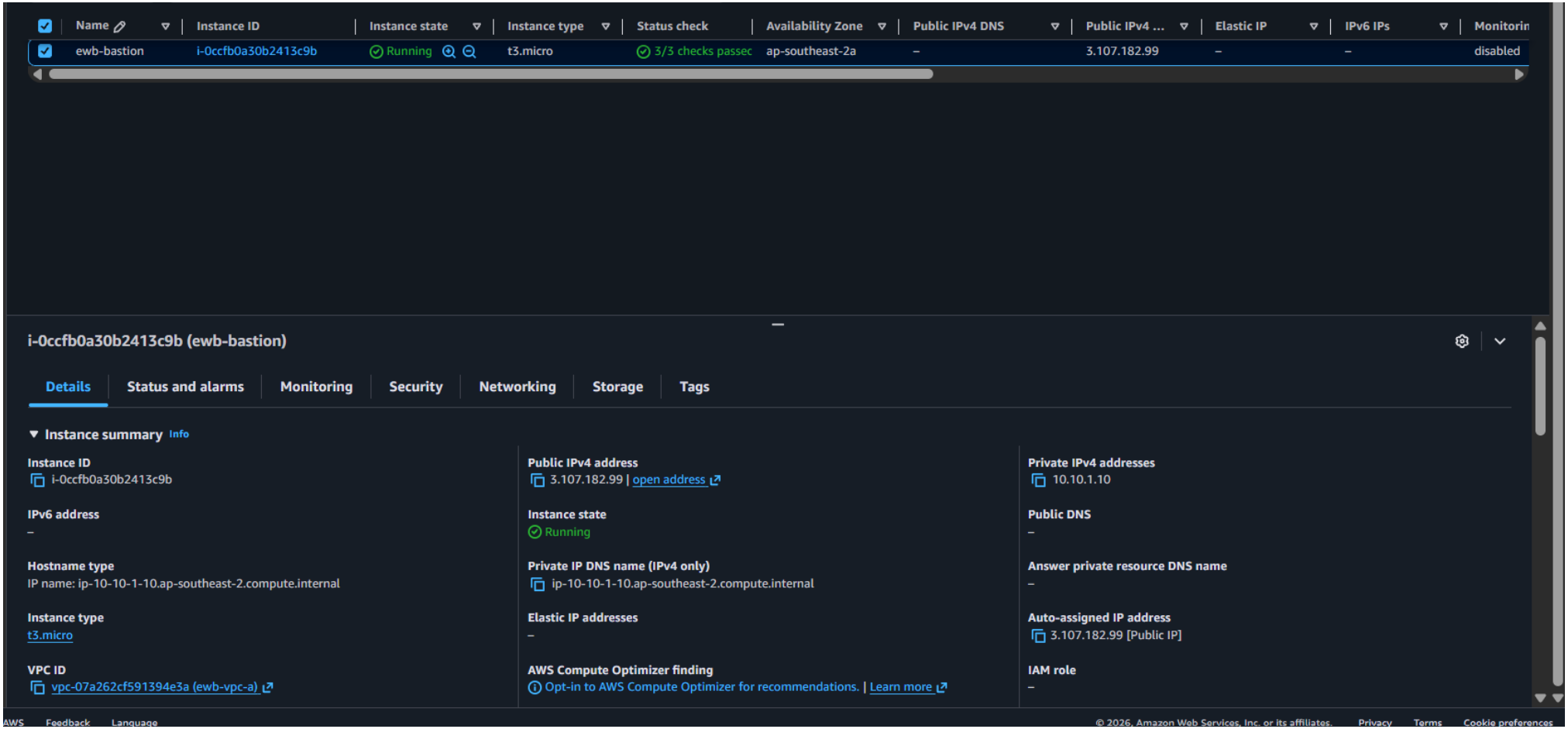


Figure evidence: image(20261106-174356).png

App-A instance details

The selected App-A instance shows private IP 10.10.2.10 and no public IPv4 address.

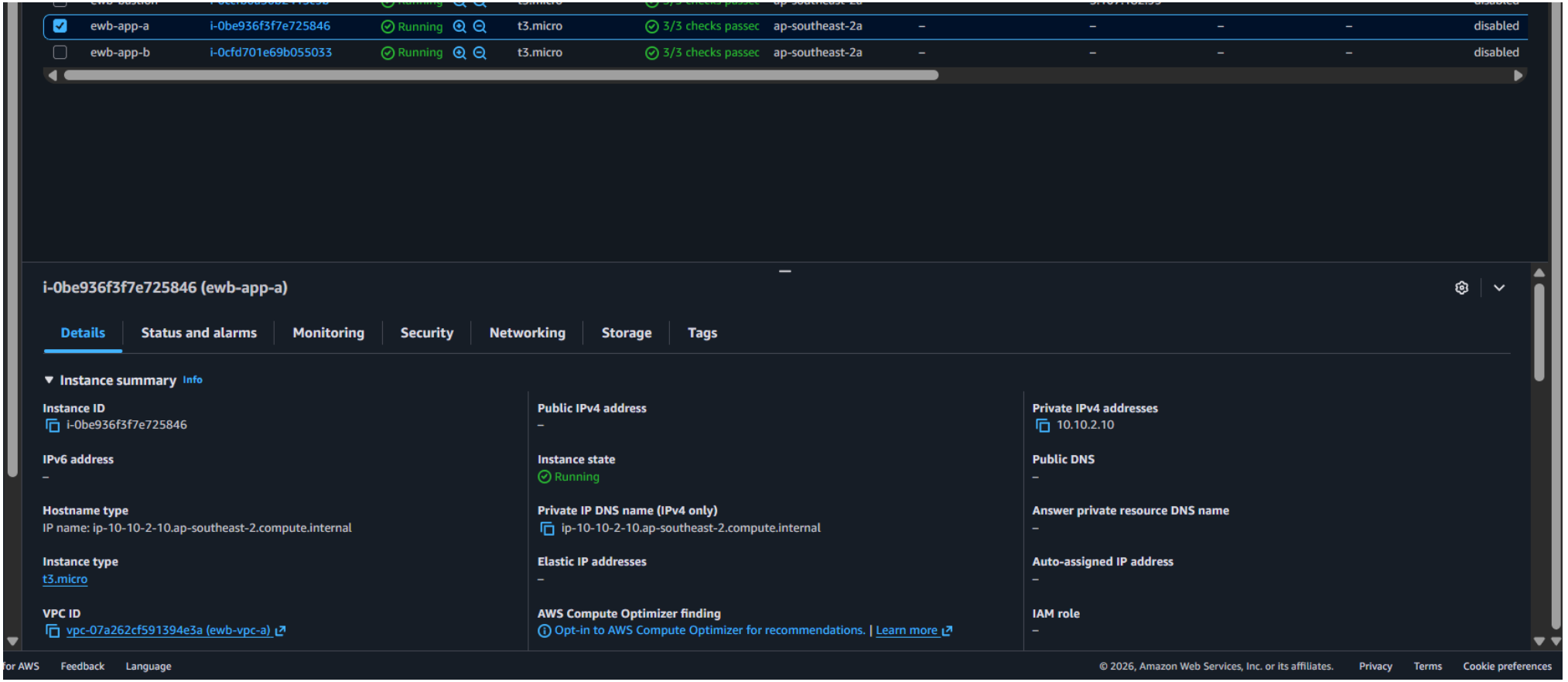


Figure evidence: image(20261107-113435).png

App-B instance details

The selected App-B instance shows private IP 10.20.1.10 and no public IPv4 address.

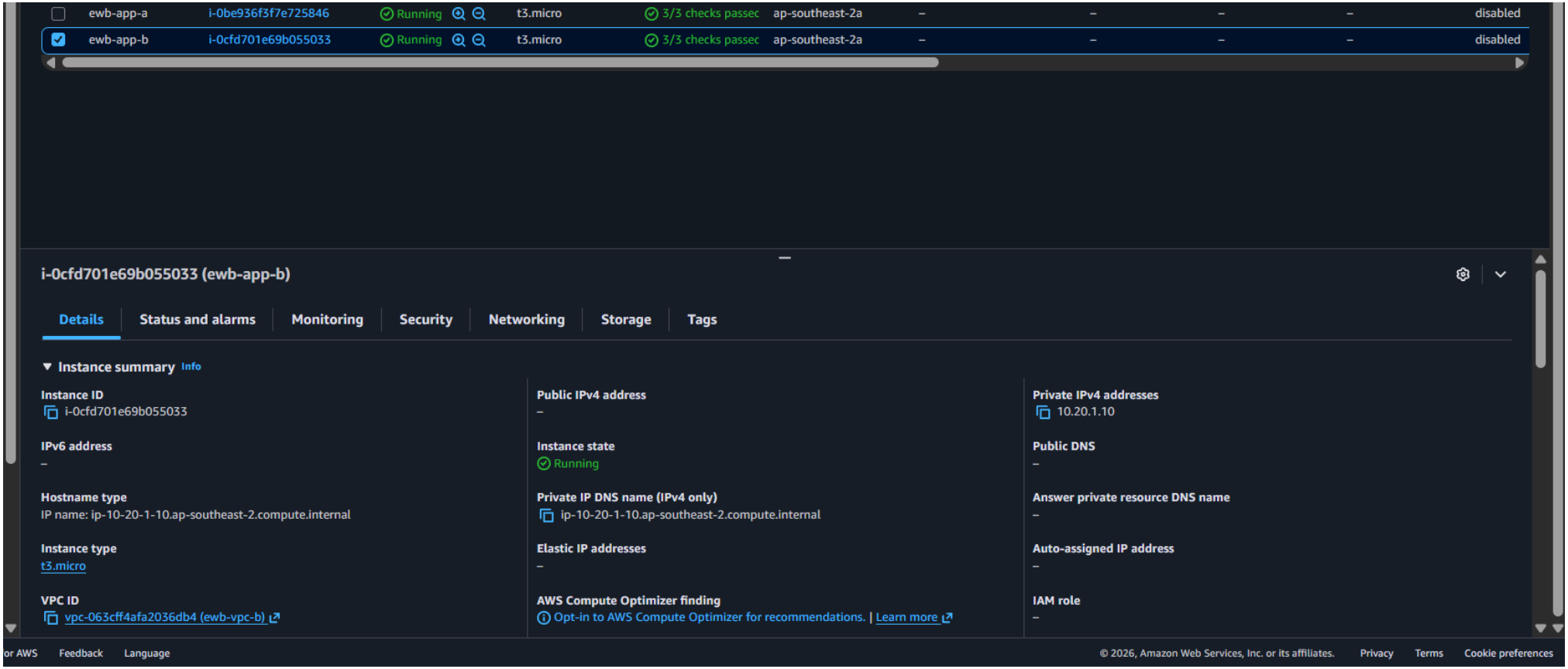


Figure evidence: image(20261107-113342).png

SSH login to Bastion

The PowerShell session reaches the Amazon Linux prompt for ip-10-10-1-10.

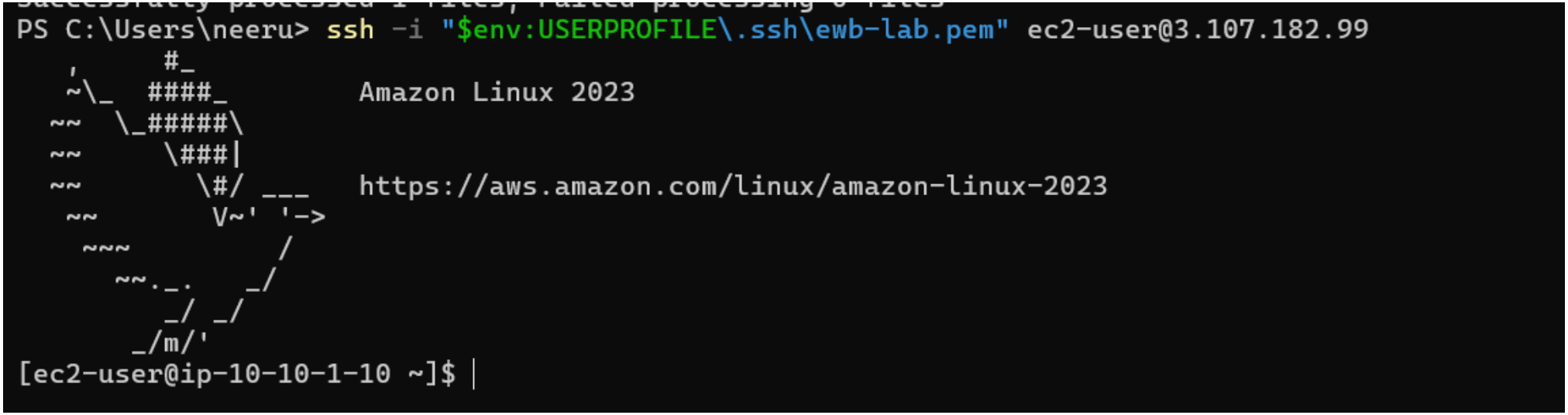


Figure evidence: image(20261107-114259).png

SSH login to App-A through Bastion

The terminal shows the App-A shell prompt at 10.10.2.10 after the SSH ProxyCommand connection.

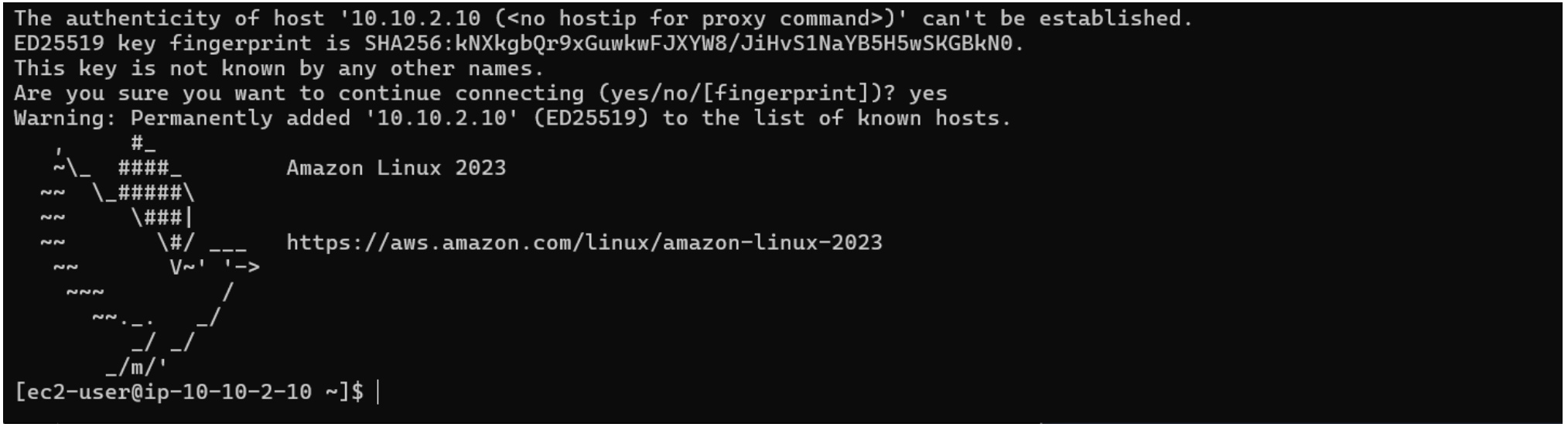


Figure evidence: image(20261107-114641).png

SSH login to App-B through Bastion

The terminal shows the App-B shell, hostname and ec2-user account.

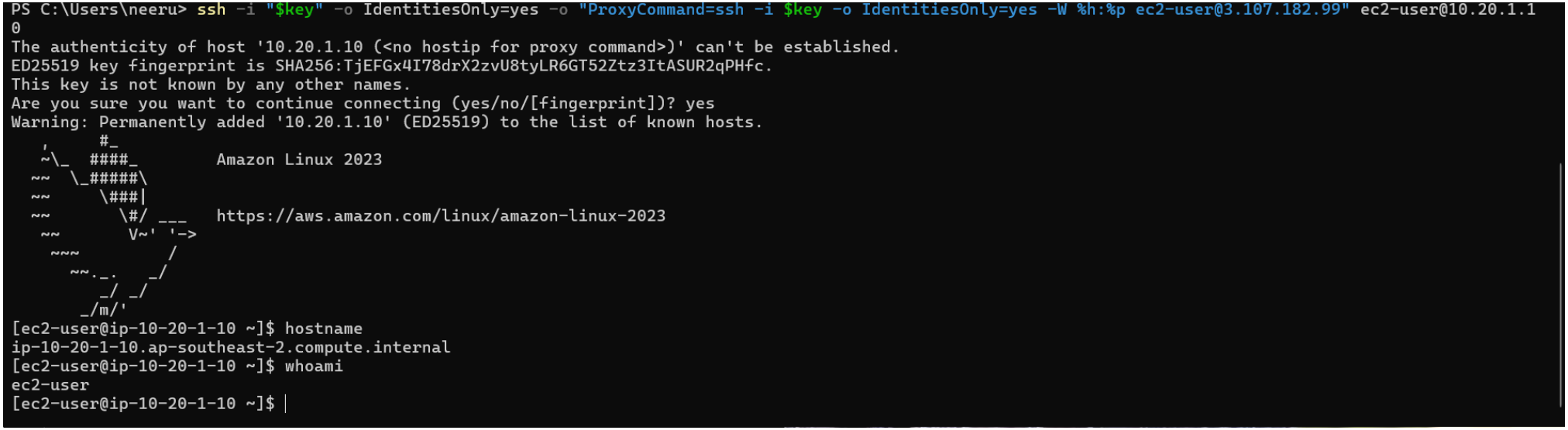


Figure evidence: image(20261107-122530).png

Ping: App-B to App-A

From App-B, ping to 10.10.2.10 returned 4/4 packets and 0% packet loss.

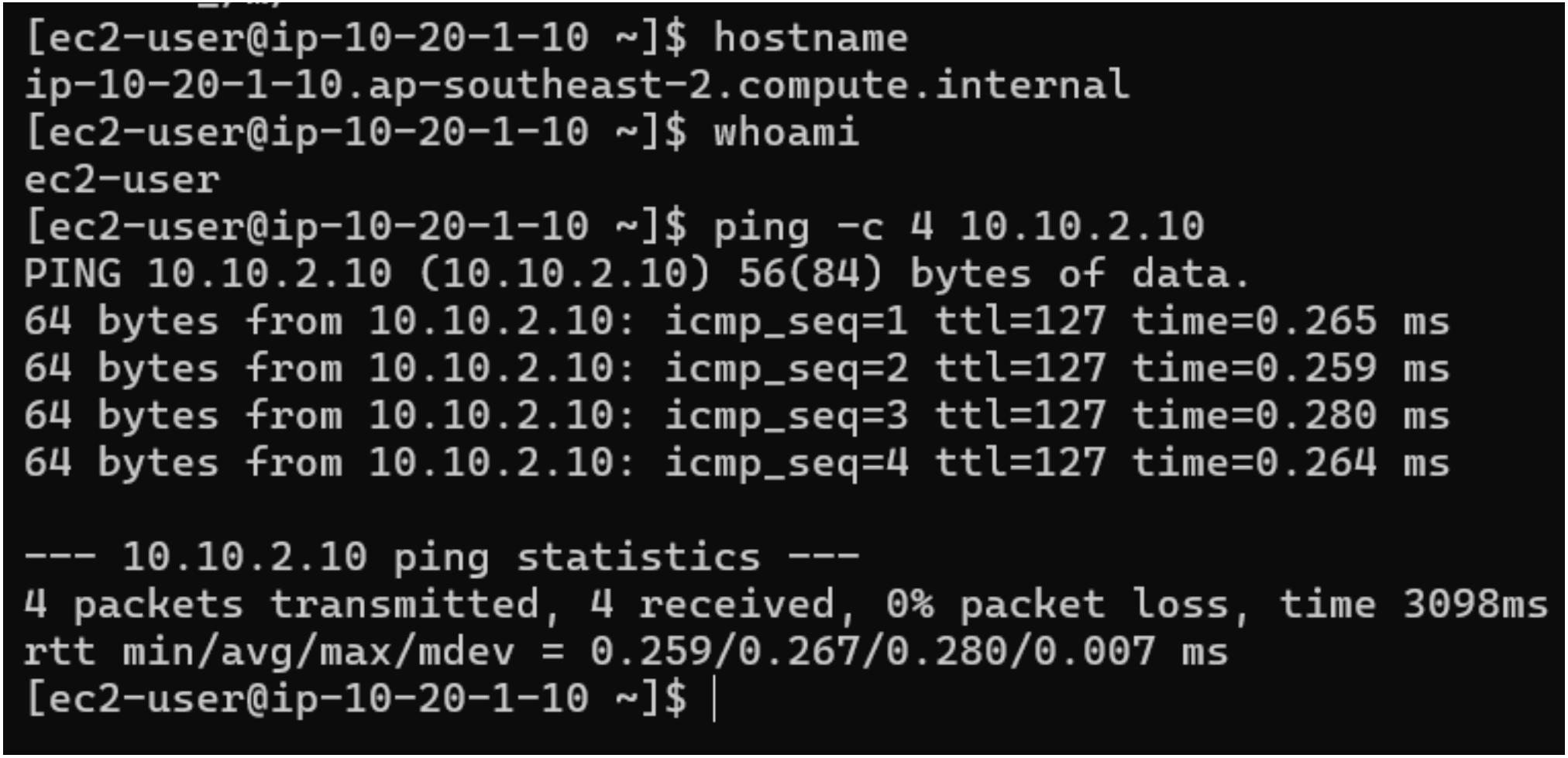


Figure evidence: image(20261107-122629).png

Ping: App-A to App-B

From App-A, ping to 10.20.1.10 returned 4/4 packets and 0% packet loss.

```
PS C:\Users\neeru> ssh -i "$key" -o IdentitiesOnly=yes -o "ProxyCommand=ssh -i $key -o IdentitiesOnly=yes -W %h:%p ec2-user@3.107.182.99" ec2-user@10.10.2.1
0
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sat Nov  7 11:46:29 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.248 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.260 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.247 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.261 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3081ms
rtt min/avg/max/mdev = 0.247/0.254/0.261/0.006 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Figure evidence: image(20261107-122816).png